ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Janice Ye

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A05_Sp26.Rmd"). Then change "Student Name" on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: "readxl", "ggplot2", "forecast","tseries", and "Kendall". Install these packages, if you haven't done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
#Load/install required package here
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(tseries)
library(ggplot2)
library(Kendall)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
library(tidyverse)  #load this package so yon clean the data frame using pipes
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr   1.1.4     v stringr 1.5.1
## v forcats 1.0.0     v tibble  3.2.1
## v purrr   1.0.2     v tidyr   1.3.1
## v readr   2.1.5

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(readxl)
```

Consider the same data you used for A04 from the spreadsheet "Table_10.1_Renewable_Energy_Production_and_Consump
The data comes from the US Energy Information and Administration and corresponds to the December 2025 Monthly Energy Review.

```r
#Importing data set - using readxl package
energy_data <- read_excel(
  path = "./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 12,
  sheet = "Monthly Data",
  col_names = FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
#Now let's extract the column names from row 11 only
read_col_names <- read_excel(
  path ="./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 10,
  n_max = 1,
  sheet ="Monthly Data",
  col_names = FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)

nobs = nrow(energy_data)
nvar = ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month               `Wood Energy Production` `Biofuels Production`
##   <dttm>                                 <dbl> <chr>
## 1 1973-01-01 00:00:00                     130. Not Available
## 2 1973-02-01 00:00:00                     117. Not Available
## 3 1973-03-01 00:00:00                     130. Not Available
## 4 1973-04-01 00:00:00                     125. Not Available
## 5 1973-05-01 00:00:00                     130. Not Available
## 6 1973-06-01 00:00:00                     125. Not Available
## # i 11 more variables: `Total Biomass Energy Production` <dbl>,
## #   `Total Renewable Energy Production` <dbl>,
## #   `Hydroelectric Power Consumption` <dbl>,
## #   `Geothermal Energy Consumption` <dbl>, `Solar Energy Consumption` <chr>,
## #   `Wind Energy Consumption` <chr>, `Wood Energy Consumption` <dbl>,
## #   `Waste Energy Consumption` <dbl>, `Biofuels Consumption` <chr>,
## #   `Total Biomass Energy Consumption` <dbl>, ...
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date, Solar Energy Consumption and Wind Energy Consumption**. Check the class of columns, you will see that they are stored are characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as "Not Available". Convert the data to numeric, the "Not Available" will became NAs.

You may either filter out the "Not Available" rows and then convert the column to numeric or convert first and then remove missing values using drop_na() (or na.omit()). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is becasue they only happen in the beginning of the series!

```
#new dataset
energy_data <- select(energy_data, "Month", "Solar Energy Consumption", "Wind Energy Consumption")

#convert columns
energy_data$Month <- as.Date(energy_data$Month)
class(energy_data$Month)
```

```
## [1] "Date"
```

```
energy_data$`Solar Energy Consumption` <- as.numeric(energy_data$`Solar Energy Consumption`)
```

```
## Warning: NAs introduced by coercion
```

```
class(energy_data$`Solar Energy Consumption`)
```

```
## [1] "numeric"
```

```
energy_data$`Wind Energy Consumption` <- as.numeric(energy_data$`Wind Energy Consumption`)
```

```
## Warning: NAs introduced by coercion
```

```
class(energy_data$`Wind Energy Consumption`)
```

```
## [1] "numeric"
```

```
#drop NAs
energy_data <- na.omit(energy_data)
```

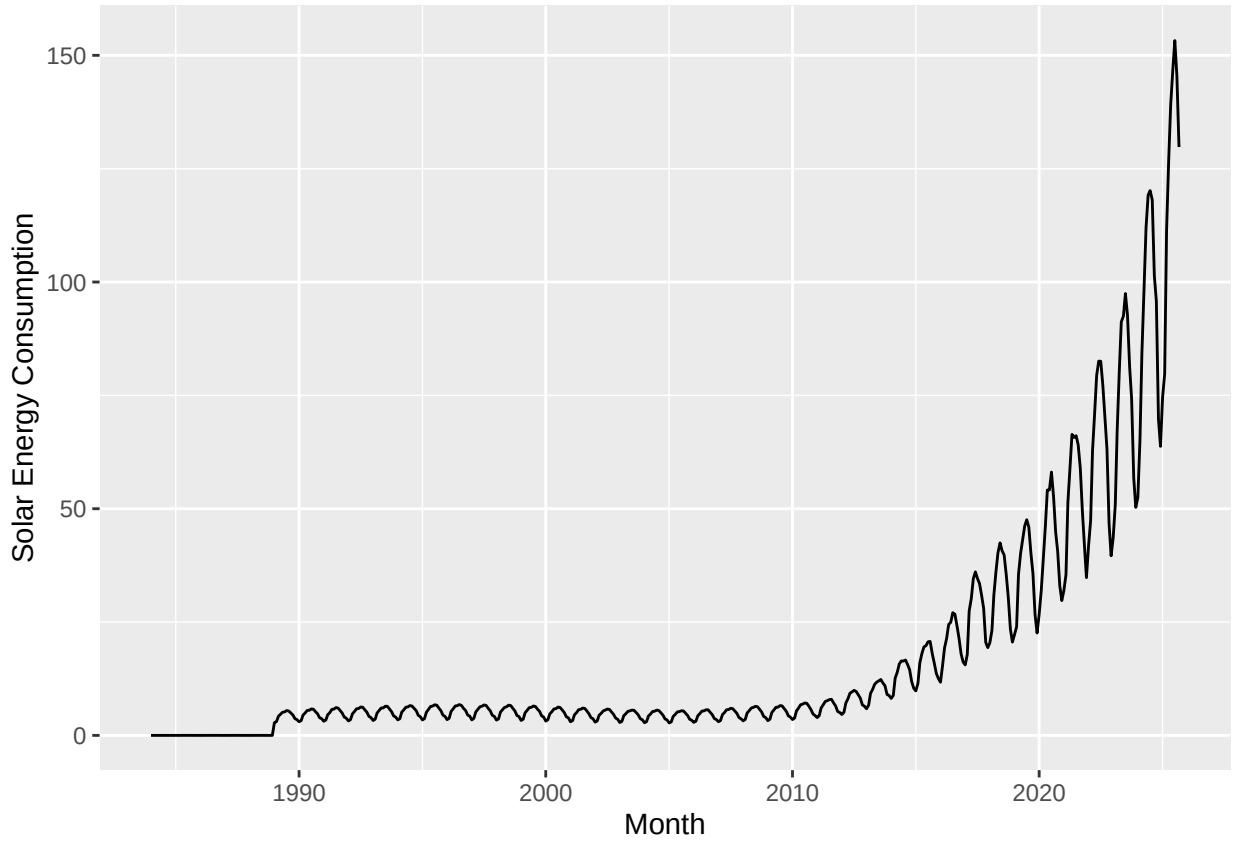
Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")")`

```
#Solar
solar_plot <- ggplot(energy_data,
                     aes(x = Month, y = `Solar Energy Consumption`)) +
  geom_line()
  ylab("Solar Energy Consumption") +
  xlab("Month") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

```
## NULL
```

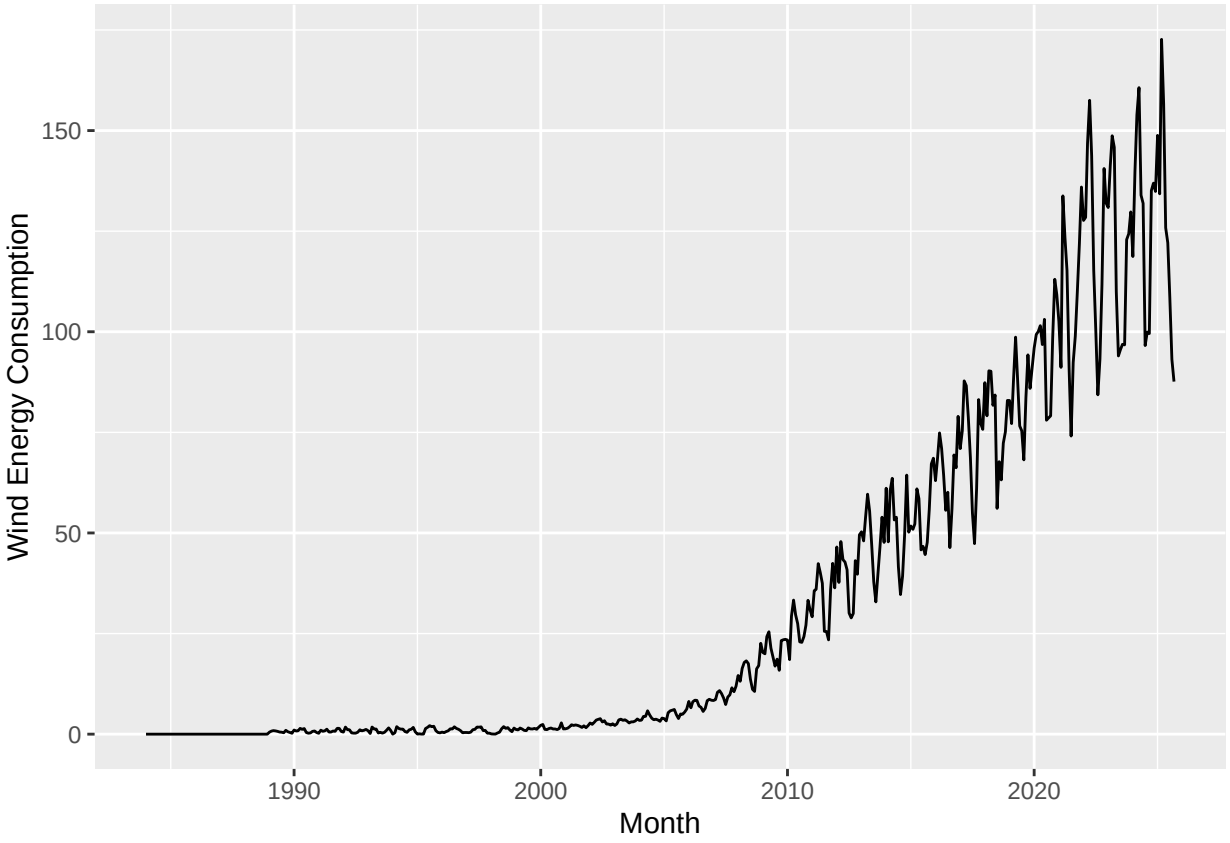
```
solar_plot
```




```
#Wind
wind_plot <- ggplot(energy_data,
                    aes(x = Month, y = `Wind Energy Consumption`)) +
  geom_line()
  ylab("Wind Energy Consumption") +
  xlab("Month") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

```
## NULL
```

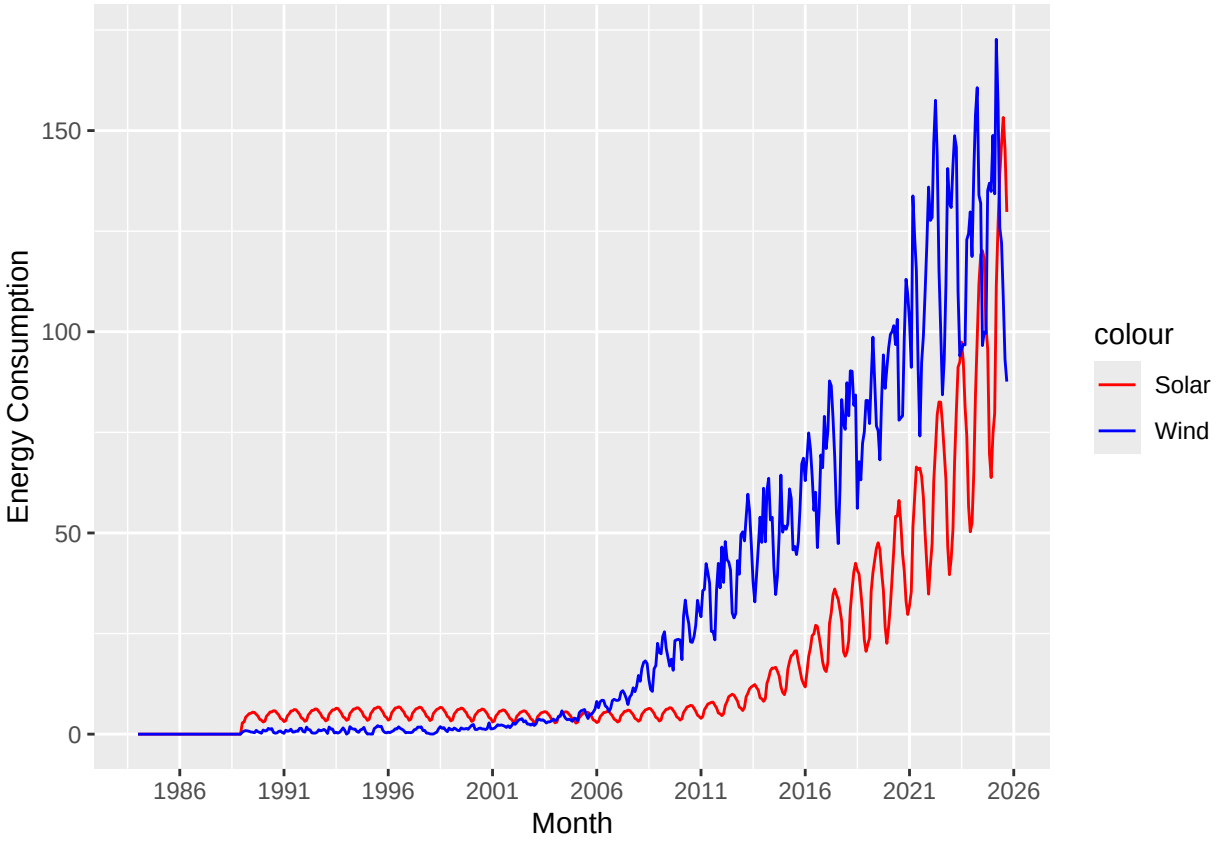
```
wind_plot
```



Q3

Now plot both series in the same graph, also using ggplot(). Use function `scale_color_manual()` to manually add a legend to ggplot. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption)`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```
combined_plot <- ggplot(energy_data, aes(x = Month)) +
  geom_line(aes(y = `Solar Energy Consumption`, color = "Solar")) +
  geom_line(aes(y = `Wind Energy Consumption`, color = "Wind")) +
  ylab("Energy Consumption") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  scale_color_manual(values = c("Solar" = "red", "Wind" = "blue"))
combined_plot
```



Decomposing the time series

The stats package has a function called decompose(). This function only take time series object. As the name says the decompose function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

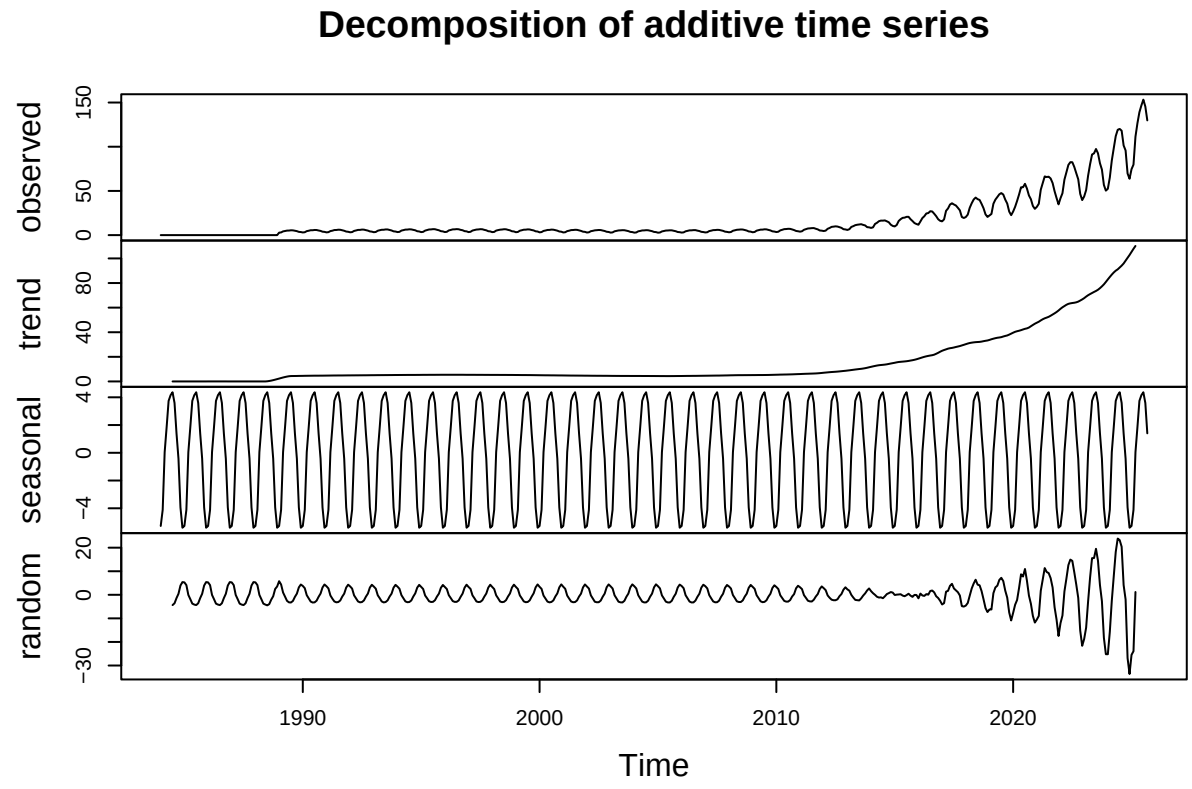
Additional info on `decompose()`.

1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
2) The trend is not a straight line because it uses a moving average method to detect trend.
3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

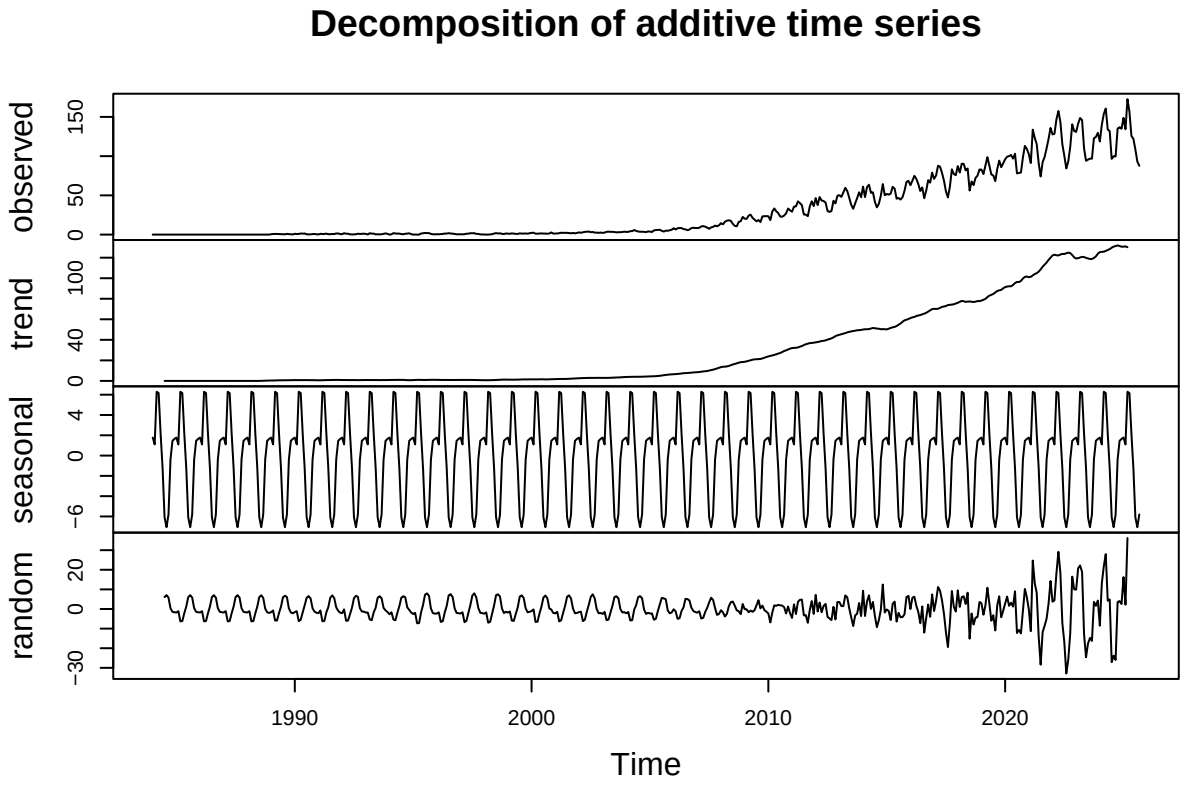
Q4

Transform wind and solar series into a time series object and apply the decompose function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```r
#transform into time series
solar_ts <- ts(energy_data$`Solar Energy Consumption`, start = c(1984,1), frequency = 12)
wind_ts <- ts(energy_data$`Wind Energy Consumption`, start = c(1984,1), frequency = 12)

#decompose
solar_decomp <- decompose(solar_ts, type = "additive")
plot(solar_decomp)
```




```r
wind_decomp <- decompose(wind_ts, type = "additive")
plot(wind_decomp)
```

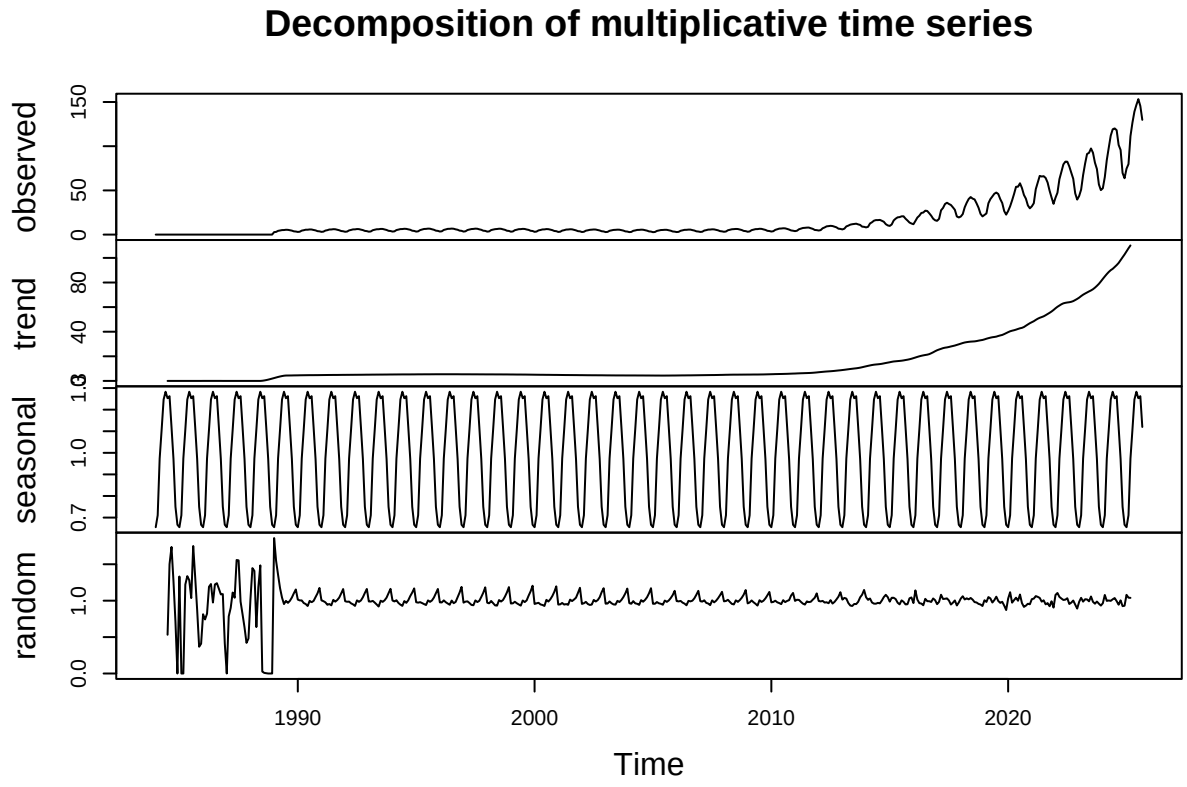


Answer: For solar, the series is nearly flat for a long time, then starts rising sharply. The oscillations (seasonality) are present throughout, but once the level rises, the swings look larger in the observed series simply because the overall scale grows. The trend is accelerating upward, indicating exponential growth. The seasonal pattern is very stable over time with roughly constant amplitude. The remainder shows increasing variance over time and has clear oscillatory structure near the end. Together, this implies that additive model of decomposition is probably not the best choice here because the variance or seasonal magnitude appears to grow with the level. For wind, the observed is very flat until the mid-2000s, then steady growth, and after 2015 it becomes highly volatile. Trend is smooth and strongly increasing after 2005. Seasonal component is stable and repeating. Random has huge late swings after 2010. Together, the series becomes more variable as it grows. And seasonality may be effectively multiplicative.

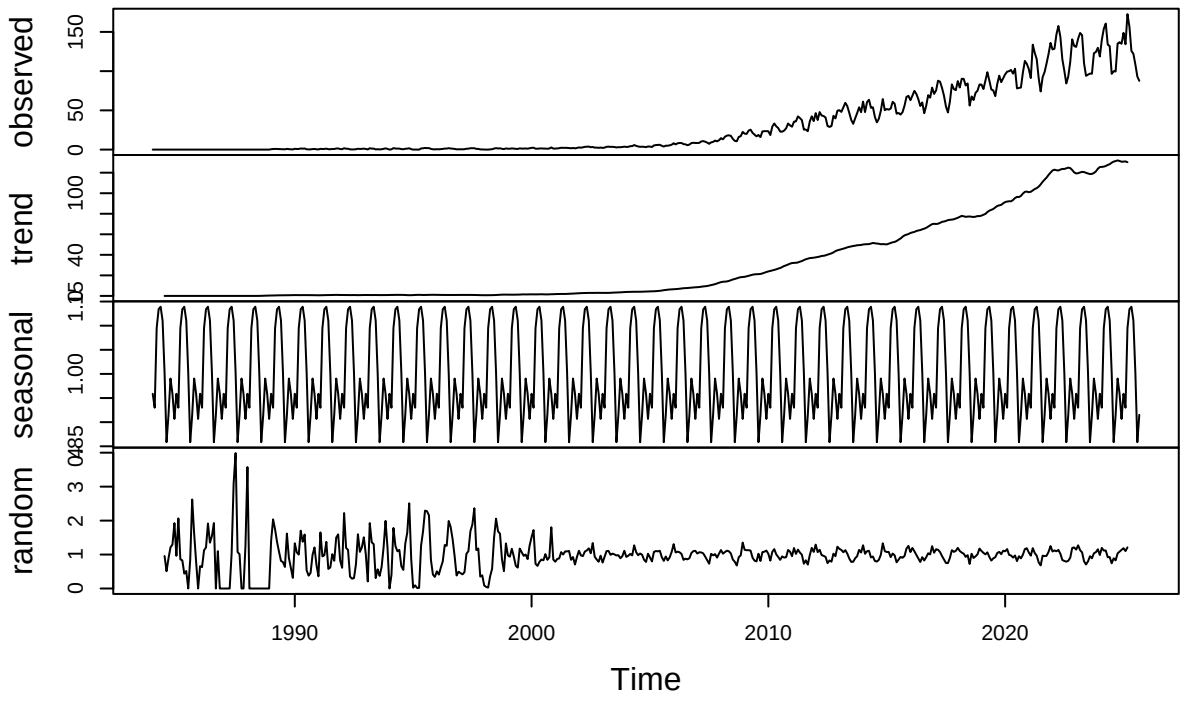
Q5

Use the decompose function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

```
#solar
solar_multi_decomp <- decompose(solar_ts, type = "multiplicative")
plot(solar_multi_decomp)
```




```r
#wind
wind_multi_decomp <- decompose(wind_ts, type = "multiplicative")
plot(wind_multi_decomp)
```



Answer: For solar, multiplicative model looks more appropriate than additive model. Random component stays tight around 1 with relatively constant variance after 1990, which means the model has absorbed the swings into seasonal or trend components, instead of leaving it as exploding residuals. For wind, multiplicative model also looks more appropriate than additive model. After 2000, the remainder stays fairly tight around 1.

Q6

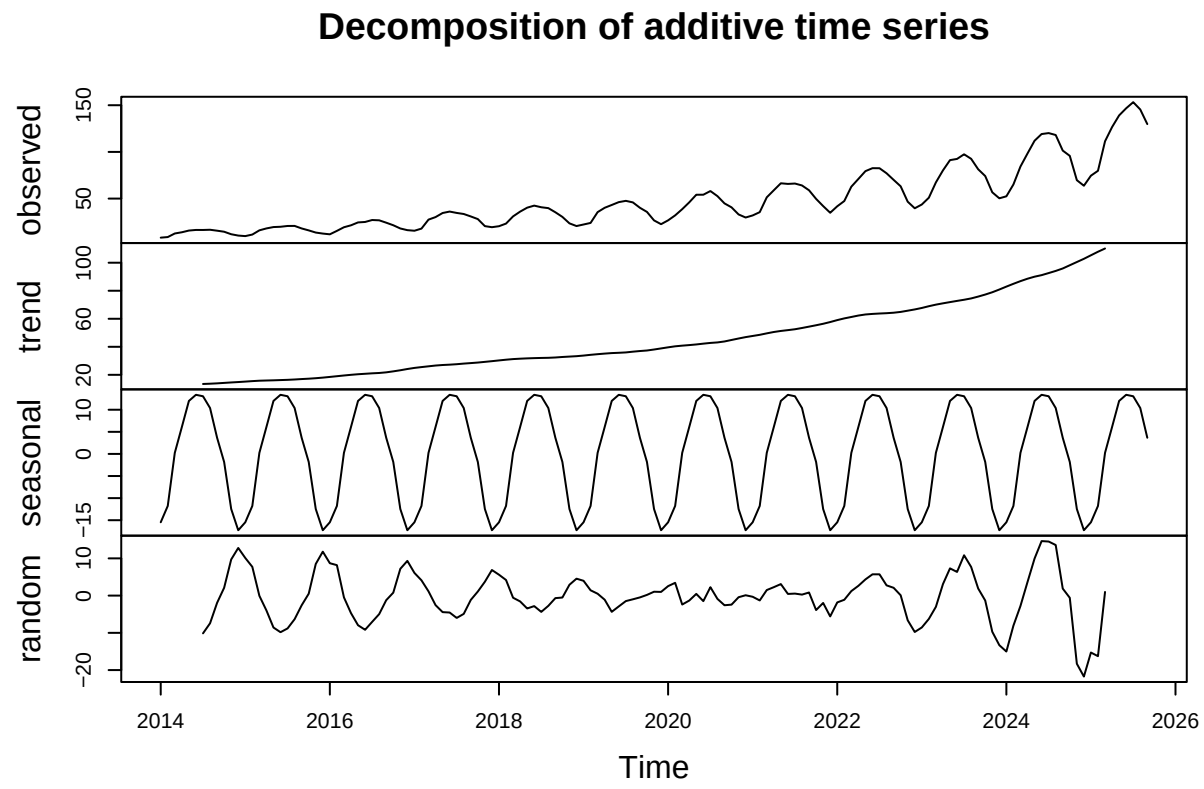
When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

Answer: Probably no because based on previous questions, we can see that trend can suddenly change a lot that does not resonate historic behaviors. For a short term forecast, the recent years contain the most relevant information. Earlier data primarily reflect outdated levels and relationships that can introduce bias and reduce forecast. accuracy.

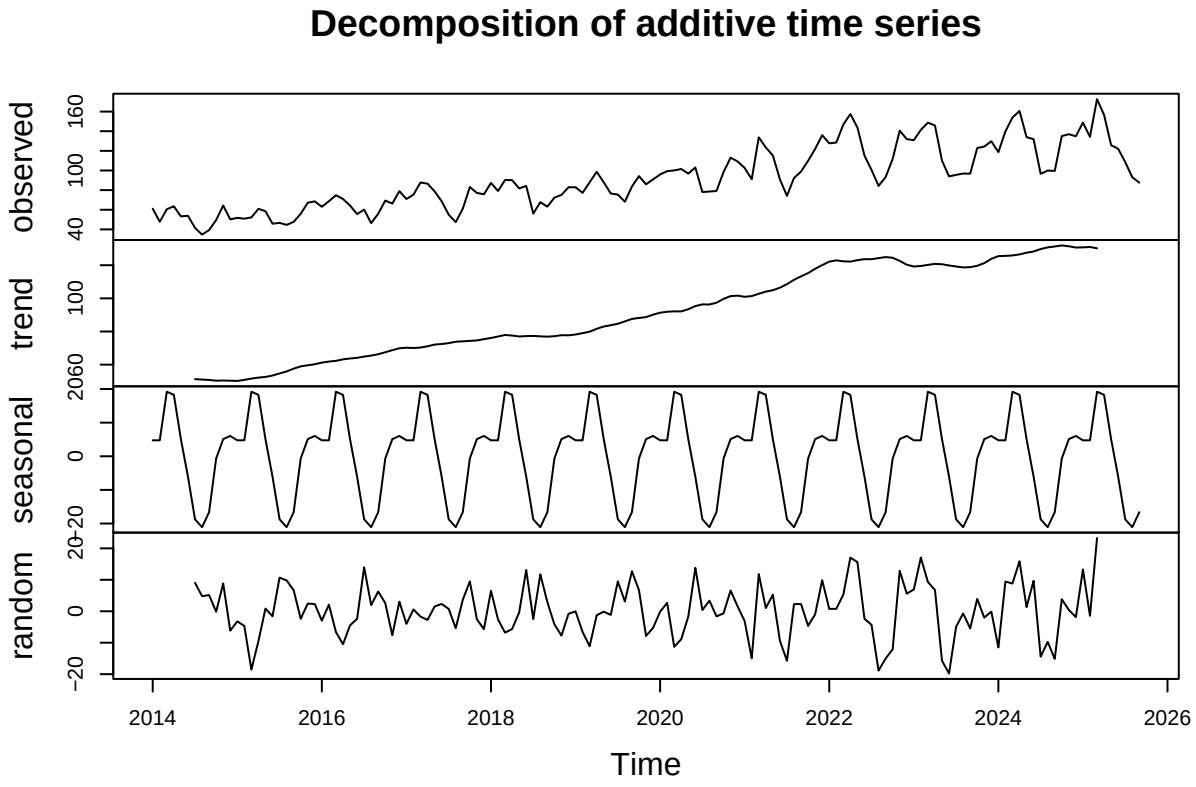
Q7

Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, .i.e, `filter(xxxx, year(Date) >= 2014 )`. Apply the decompose function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```r
energy_data_new <- filter(energy_data, year(Month) >= 2014)

#solar
solar_2014_ts <- ts(energy_data_new[,2], start = c(2014,1), frequency = 12)

solar_2014_decomp <- decompose(solar_2014_ts, type = "additive")
plot(solar_2014_decomp)
```




```r
#wind
wind_2014_ts <- ts(energy_data_new[,3], start = c(2014,1), frequency = 12)

wind_2014_decomp <- decompose(wind_2014_ts, type = "additive")
plot(wind_2014_decomp)
```



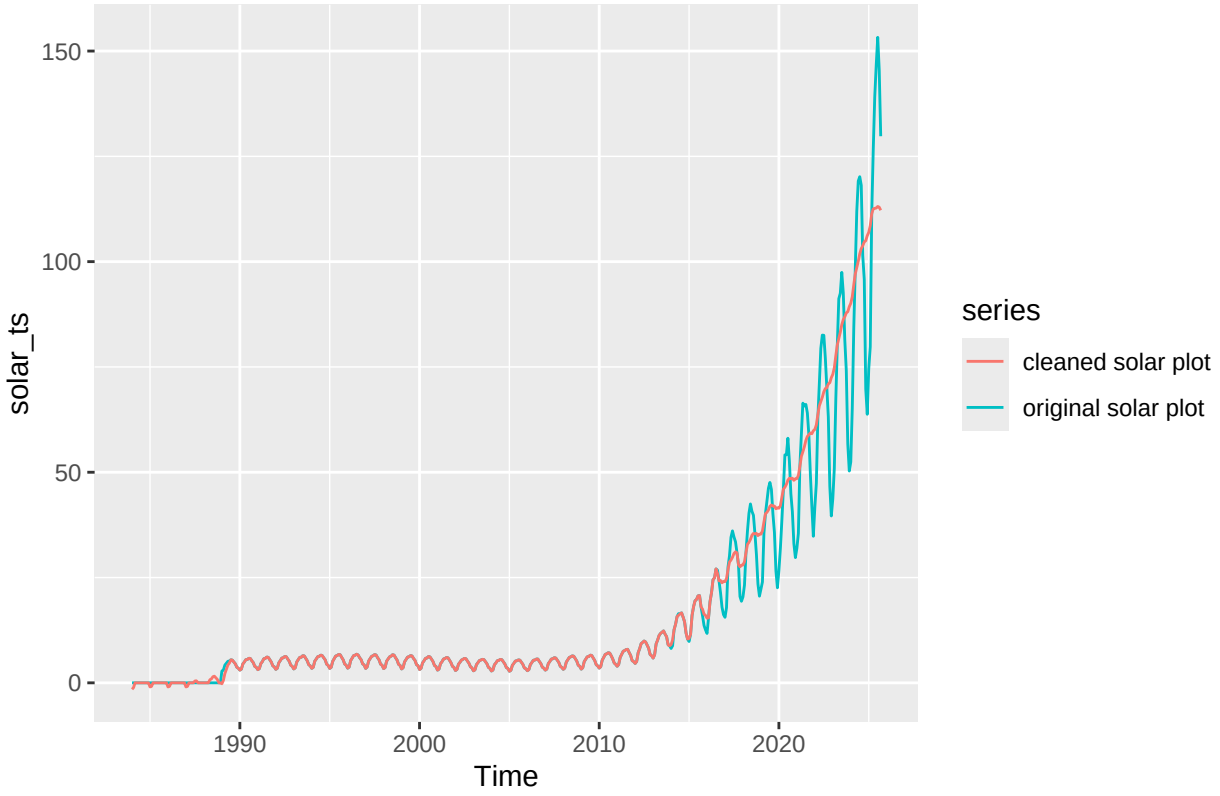
Answer: For solar, observed component has a upward trend with strong repeating seasonal swings. Trend component is smooth, steadily increasing with faster growth near the end. Seasonal component is very consistent, and random component does not look completely random. It has clusters and some bigger deviations. This suggests that there might be shocks or that the trend is changing too fast for classical decomposition to fully capture. For wind, observed component shows upward movement. Trend component rises steadily, and seasonal component is very consistent. Random component looks more random than solar.

Identify and Remove outliers

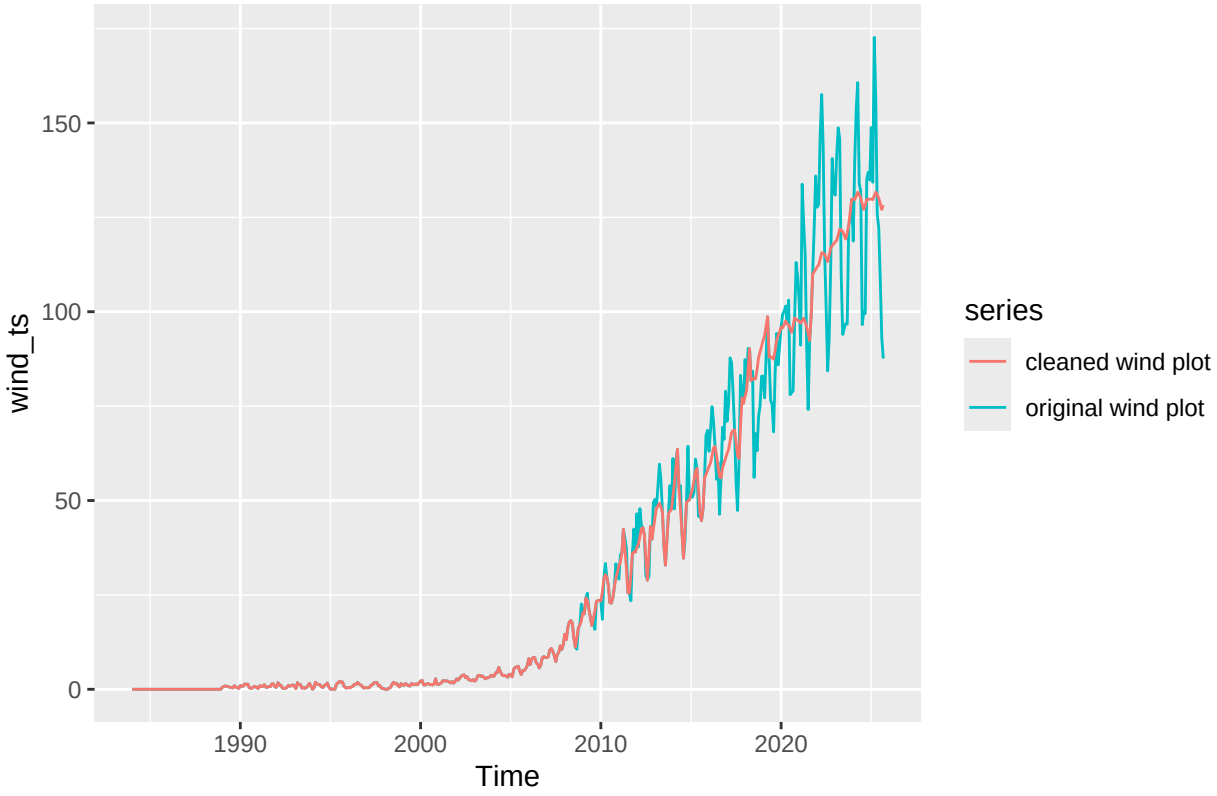
Q8

Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

```
#solar
solar_clean_ts <- tsclean(solar_ts)

solar_compare_plot <- autoplot(solar_ts, series = "original solar plot") +
  autolayer(solar_clean_ts, series = "cleaned solar plot")
solar_compare_plot
```




```
#wind
wind_clean_ts <- tsclean(wind_ts)

wind_compare_plot <- autoplot(wind_ts, series = "original wind plot") +
  autolayer(wind_clean_ts, series = "cleaned wind plot")
wind_compare_plot
```

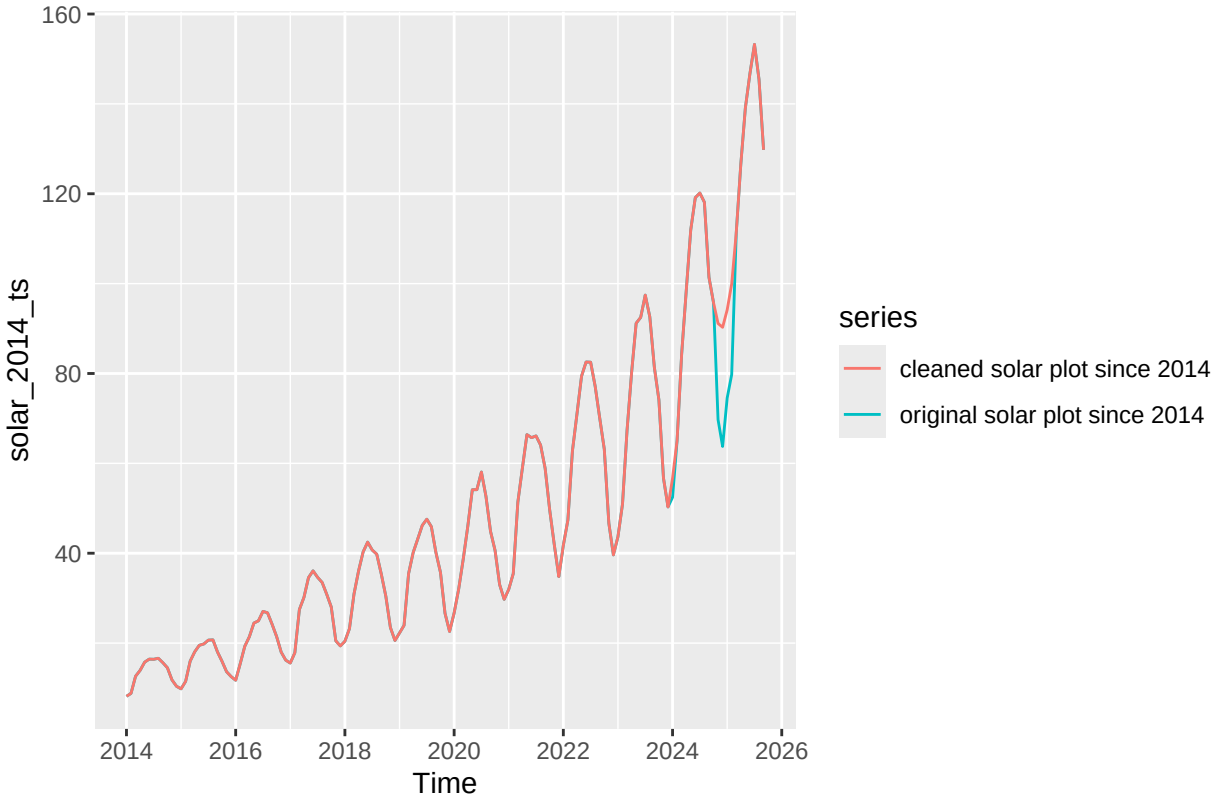


Answer: For solar, outliers are removed because based on the comparison plot, originial plot has more up and downs while cleaned plot smoothed out these spikes. Similar pattern applies to wind, with cleand plot smoothed out the ups and downs compared to the original plot.

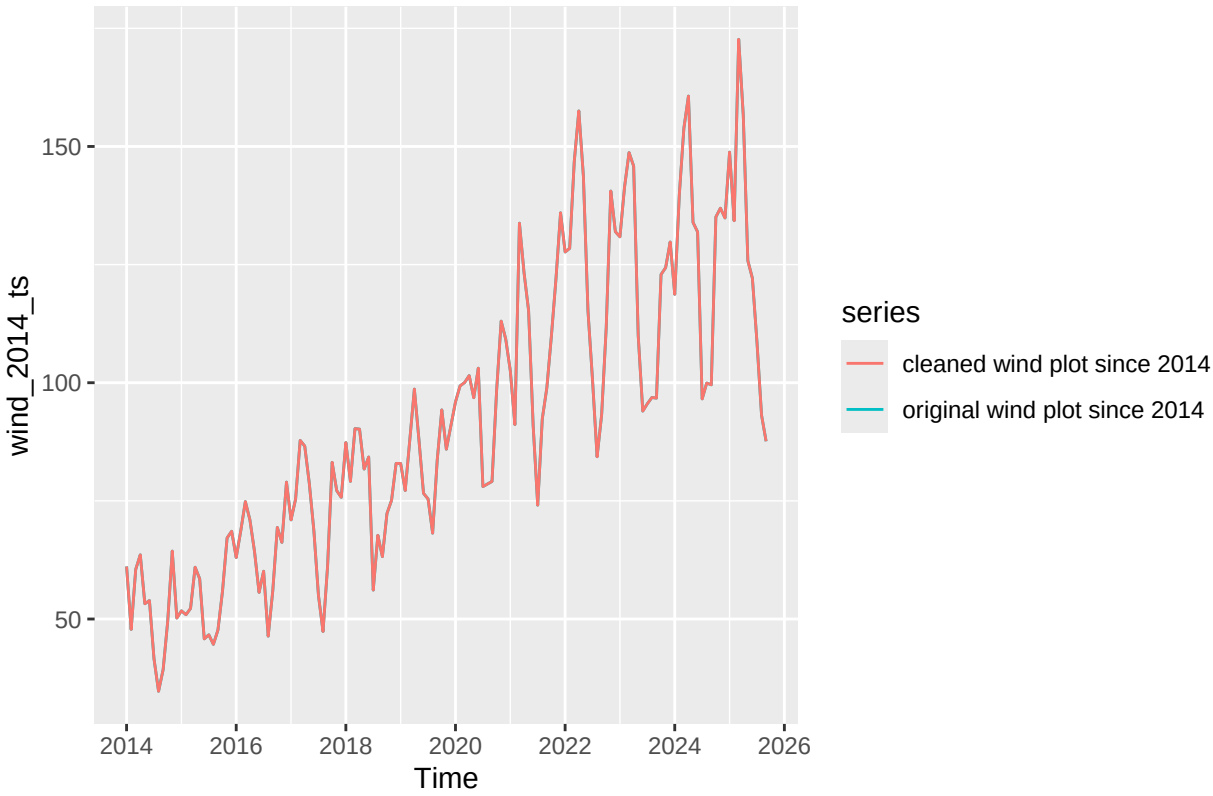
Q9

Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what `autoplot()` again what happened now? Did the function removed any outliers from the series?

```
#solar
solar_2014_clean <- tsclean(solar_2014_ts)

solar_compare_2014_plot <- autoplot(solar_2014_ts, series = "original solar plot since 2014") +
  autolayer(solar_2014_clean, series = "cleaned solar plot since 2014")
solar_compare_2014_plot
```




```
#wind
wind_2014_clean <- tsclean(wind_2014_ts)

wind_compare_2014_plot <- autoplot(wind_2014_ts, series = "original wind plot since 2014") +
  autolayer(wind_2014_clean, series = "cleaned wind plot since 2014")
wind_compare_2014_plot
```



Answer: For solar, it is very clear that cleaned plot removed a down spike around 2025, showing it effectively removed outliers. For wind, cleaned and originial plot exactly overlap which means it did not have any outlier or the function did not removed any.